HTOC

HEALTHCARE THREAT OPERATIONS CENTER

Tipper — Daily Partner Indicator Tips

Version 1.0
April 2026

Table of Contents

Table of Contents

Table of Contents 2

1. Purpose 3

2. Scope 3

3. Prerequisites 3

3.1 Environment 3

3.2 Configuration File 4

3.3 Input Data 4

4. Key Configuration 5

5. Pipeline Overview 6

6. Step-by-Step Logic 7

Step 1 — Initialize ThreatConnect 7

Step 2 — Load OpDiv Observation CSVs 7

Step 3 — Query ThreatConnect 7

Step 4 — Process Recent Tags 8

Step 5 — Exclude SOAR-Created Indicators 8

Step 6 — Enrich with VirusTotal and Shodan 9

Step 7 — Filter by VT Malicious Score 10

Step 8 — Finalize and Rename Columns 10

Step 9 — Bucket by Partner 11

Step 10 — Save Excel Workbook 11

7. Batch Launcher (run_tipper.bat) 12

Execution sequence: 12

Log files: 13

8. Output File 13

9. Troubleshooting 14

10. Relationship to Other Processes 15

11. Related Documents 16

1. Purpose

The Tipper script generates daily threat indicator tips for HTOC partner organizations. It pulls recently observed indicators from ThreatConnect, cross-references them against OpDiv observation data, filters to high-confidence malicious indicators (VirusTotal score > 10), enriches them with Shodan and VirusTotal data, and outputs a partner-bucketed Excel workbook — one tab per partner — to the shared Tippers directory.

The output gives each partner a targeted, enriched view of threat indicators actively observed in their environment within the last 24 hours that meet a minimum malicious threshold.

-

2. Scope

This procedure applies to HTOC analysts and data engineers who run, monitor, or troubleshoot the daily Tipper process. It covers the full pipeline from ThreatConnect query through Excel output, as well as the batch launcher and its logging behavior.

-

3. Prerequisites

3.1 Environment

Requirement	Notes
Python 3.13	Located at `C:\Program Files\Python313\python.exe`

Requirement	Notes
pandas, pytz, numpy, xlsxwriter, openpyxl	Installed automatically by `run_tipper.bat` at each execution
ThreatConnect SDK	Located at `\\10.1.4.22\data\HTOC\Data_Analytics\threatconnect`
Access to `\\10.1.4.22\data\HTOC\`	Input OpDiv CSVs and output Tippers folder are on this UNC path
ThreatConnect API credentials	Stored in `utils\config.json` (see Section 4)

Example dependency install command:

```
pip install pandas
```

> **Note:** The script accesses `\\10.1.4.22\` via UNC path directly, so the `Z:\` drive does not need to be mapped for the automated run.

3.2 Configuration File

API credentials are loaded from:

```
scripts/batch-processing-script/Tipper/utils/config.json
```

This file is read by `utils/config_loader.py` and must contain valid `access_id`, `secret_key`, `default_org`, and `base_url` values for ThreatConnect. Contact the HTOC admin if credentials have rotated.

3.3 Input Data

Source	Path
OpDiv observation CSVs	`\\10.1.4.22\data\HTOC\Data_Analytics\Data\OpDiv_Observations\htoc_opdiv_obs_d{YYYYMMDD}.csv`

Source	Path
ThreatConnect indicators	Queried live via API — no local file

OpDiv CSV files for the last **3 days** must be present before the script runs. These are generated by the upstream OpDiv observation feed.

-

4. Key Configuration

All path constants are defined at the top of `main.py`:

Constant	Value
`SDK_PATH`	`\\10.1.4.22\data\HTOC\Data_Analytics\threatconnect`
`CONFIG_PATH`	`...\Tipper\utils\config.json`
`TIPPERS_PATH`	`\\10.1.4.22\data\HTOC\HTOC Reports\Tippers`
`BASE_PATH`	`\\10.1.4.22\data\HTOC\Data_Analytics\Data\OpDiv_Observations\htoc_opdiv_obs_d{date}.csv`
`DATE_FORMAT`	`%Y%m%d`

Key behavioral parameters set in function logic:

Parameter	Value	Description
TC query lookback	3 days	Indicators with `lastObserved` within the last 3 days
OpDiv file lookback	3 days	Loads observation CSVs for the last 3 days
Observation recency filter	24 hours	Only indicators observed in the last 24 hours pass through
VT malicious threshold	> 10	Indicators with `vtMaliciousCount` ≤ 10 are excluded
TC owner	`HTOC Org`	Only indicators from this ThreatConnect owner are queried

-

5. Pipeline Overview

```
[1] Initialize ThreatConnect session
        ↓
[2] Load OpDiv observation CSVs (last 3 days)
        ↓
[3] Query ThreatConnect for recently observed indicators (last 3 days)
        ↓
[4] Process recent tags — match indicators to partner API tags, filter to last 24h
        ↓
[5] Exclude SOAR-created indicators (per-indicator attribute lookup)
        ↓
[6] Enrich with VirusTotal and Shodan
        ↓
[7] Filter to VT malicious count > 10
        ↓
[8] Extract and flatten enrichment fields
        ↓
[9] Bucket indicators by partner
        ↓
```

```
[10] Save Excel workbook — one tab per partner
```

-

6. Step-by-Step Logic

Step 1 — Initialize ThreatConnect

`initialize_threatconnect()` reads credentials from `config.json`, instantiates the `ThreatConnect` SDK object, and creates a global `RequestObject` pre-configured with HTTP `GET` and owner `HTOC Org`. On failure, the script prints an error and exits with code 1.

-

Step 2 — Load OpDiv Observation CSVs

`get_file_paths()` generates file paths for the last **3 days** using UTC dates, then filters to only paths that exist on disk. `load_observed_data()` reads each file and concatenates them into a single DataFrame (`observed_data_df`). Missing files are skipped with an error message.

-

Step 3 — Query ThreatConnect

`query_observed_src()` queries the ThreatConnect `/v3/indicators` endpoint using TQL for:

```
ownerName EQ "HTOC Org"
```

`typeName IN` (Address, EmailAddress, File, Host, URL, ASN, CIDR, Email Subject, Hashtag, Mutex, Registry Key, Stripped URL, User Agent)

```
lastObserved >= {3 days ago}T00:00:00Z
```

Fields retrieved: `tags`, `observations`. Results are normalized, deduplicated by `indicator`, and filtered to indicators with `lastObserved` within the 3-day window. The result is stored as `observed_src`.

If `observed_src` is empty, the script exits early.

-

Step 4 — Process Recent Tags

`process_recent_tags()` identifies partner-linked indicators by:

Iterating over `observed_src` and exploding `tags.data` for each indicator.

Normalizing the tag name `VA CSOC CTS Splunk` → `VA Splunk API`.

Keeping only indicators that have at least one tag containing `"API"` (case-insensitive) — these are the partner Splunk API tags.

Inner-joining with `observed_data_df` on `summary` + `OpDiv`.

Filtering to indicators where both `lastObserved` and `obs_date` fall within the **last 24 hours**.

Extracting `partner` from the tag name (strips `" Splunk API"` suffix).

Aggregating partner counts and names per indicator into `partner_count` and `Partners` columns.

Deduplicating to one row per indicator.

If no matches are found, the script exits early.

-

Step 5 — Exclude SOAR-Created Indicators

`excludeSoarData()` calls the ThreatConnect `/v3/indicators/{indicator}?fields=attributes` endpoint for each indicator in `recent_tags`. It:

Collects attribute metadata including `createdBy.lastName`.

Excludes any indicator whose attributes were created by `lastName == 'SOAR'` — these are automated SOAR-generated entries, not analyst-confirmed threats.

Indicators with **no attributes at all** are retained (they are not considered SOAR-generated).

HTTP 400 errors are silently suppressed (unsupported indicator types).

This step also triggers the **enrichment process** (Step 6) for the filtered set.

-

Step 6 — Enrich with VirusTotal and Shodan

For each remaining indicator, a POST request is sent to:

`/v3/indicators/{encoded_value}/enrich?type=Shodan&type=VirusTotalV3`

Enrichment data is flattened and merged back onto `filtered_recent_tags`. Shodan fields extracted:

Field	Column
`hostNames`	`shodan_hostNames`
`domains`	`shodan_domains`
`tags`	`shodan_tags`
`country`	`shodan_country`
`city`	`shodan_city`
`isp`	`shodan_isp`

Field	Column
`asn`	`shodan_asn`
`org`	`shodan_org`
`openPorts`	`shodan_openPorts`

VirusTotal field extracted: `vtMaliciousCount`

Shodan 400 errors for unsupported indicator types are silently suppressed.

-

Step 7 — Filter by VT Malicious Score

Only indicators with `vtMaliciousCount > 10` are retained. This threshold ensures the output contains only high-confidence malicious indicators. Indicators without VirusTotal data or with a low score are dropped.

-

Step 8 — Finalize and Rename Columns

The enrichment fields are extracted via `extract_enrichment()` and joined back. Columns are renamed to analyst-friendly names:

Internal Name	Output Column
`indicator`	`Indicator`
`vtMaliciousCount`	`Malicious Score/Count`
`obs_date`	`Observation Date`

Internal Name	Output Column
`shodan_asn`	`ASN`
`rating`	`ThreatAssessRating`
`confidence`	`ThreatAssessConfidence`
`shodan_city`	`City`
`shodan_country`	`Country`
`data.legacyLink`	`ThreatConnect Link`
`partners`	`Partners`

Missing values are filled with `"unknown"`. Duplicates are dropped.

-

Step 9 — Bucket by Partner

`all_partners` is derived from the `Partners` column (comma-separated values). A `partner_buckets` dictionary is built mapping each partner name to its subset of `filtered_recent_tags` using whole-word regex matching.

If no partners are found, the script exits without writing output.

-

Step 10 — Save Excel Workbook

`save_to_excel()` writes one Excel worksheet per partner to:

`\\10.1.4.22\data\HTOC\HTOC Reports\Tippers\tippers_by_partner_{YYYYMMDD}.xlsx`

Each sheet:

Is named using the partner name (sanitized to alphanumeric***/underscore/hyphen***, max 31 characters per Excel limit).

Has **autofilter** enabled on all columns.

Has the **header row frozen** (freeze panes at row 1).

Has **auto-sized column widths** (capped at 50 characters).

Strips timezone info from datetime columns to ensure Excel compatibility.

-

7. Batch Launcher (run_tipper.bat)

The batch file `run_tipper.bat` is the entry point used by Task Scheduler. It runs automatically every day at **7:00 AM** via **Windows Task Scheduler** on **F.R.E.D**. It wraps `main.py` with dependency checks and structured logging.

Manual execution is only necessary when troubleshooting a failed scheduled run or re-running after correcting an upstream data issue.

Execution sequence:

Writes `[` to `TipperRunLogs.json` — initializes a fresh JSON array, overwriting the previous run's log.

Logs a `Start` event with a UTC timestamp.

Runs `pip install --upgrade --quiet --user` for: `pandas pytz numpy requests xlsxwriter openpyxl`. Output appended to `TipperRunOutput.log`. If pip fails (non-zero exit code), logs a `PipInstallFailed` event and exits.

Runs `main.py`. All stdout/stderr is appended to `TipperRunOutput.log`.

Logs an `End` event with a UTC timestamp and the script's exit code.

Closes the JSON array with `]`.

Log files:

File	Location	Contents
`TipperRunLogs.json`	`Tipper\TipperRunLogs.json`	Structured JSON log: Start / PipInstallFailed / End events with timestamps and exit codes
`TipperRunOutput.log`	`Tipper\TipperRunOutput.log`	Full stdout/stderr from pip install and `main.py` (appended each run)

> **Important:** `TipperRunLogs.json` is **overwritten** at the start of every run, so it only reflects the most recent execution. `TipperRunOutput.log` is **appended** and will grow over time — archive or truncate periodically.

-

8. Output File

Detail	Value
Location	`\\10.1.4.22\data\HTOC\HTOC Reports\Tippers\`
Filename	`tippers_by_partner_{YYYYMMDD}.xlsx`
Sheets	One per partner (OpDiv name)
VT filter	Only indicators with `vtMaliciousCount > 10`
Date basis	UTC date at time of execution

-

9. Troubleshooting

Symptom	Likely Cause	Resolution
`TipperRunLogs.json` shows `PipInstallFailed`	Network/proxy blocked pip download	Run the pip install command manually from a terminal to confirm connectivity; packages may already be installed — check if `main.py` can be run directly
`[ERROR] Failed to initialize ThreatConnect`	Invalid or expired API credentials	Update `utils\config.json` with current credentials; verify ThreatConnect is reachable from F.R.E.D
`[DEBUG] observed_src is empty`	No indicators found in TC for the 3-day window	Check ThreatConnect for indicator activity; verify the query owner (`HTOC Org`) and lookback are correct
`[DEBUG] recent_tags is empty after processing`	No indicators matched to partner API tags in the last 24 hours	Verify OpDiv CSVs for the target dates exist; check that partner API tags are being applied in ThreatConnect
`[DEBUG] filtered_recent_tags is empty`	All indicators were either SOAR-created or had VT score ≤ 10	Normal on low-activity days; review SOAR exclusion criteria or lower the VT threshold if needed
`[DEBUG] all_partners is empty`	No partners in `Partners` column after filtering	Confirms no qualifying indicators for any partner today; no output file will be written

Symptom	Likely Cause	Resolution
Excel file missing for today	Script exited early (empty data at any stage)	Review `TipperRunOutput.log` for the `[DEBUG]` message that triggered the early exit
`TipperRunOutput.log` is very large	Log is append-only and accumulates across runs	Truncate or archive the file periodically; do not delete — it is the primary debug record
Shodan data missing for some indicators	Shodan does not support that indicator type (e.g., URL, Hash)	Expected behavior; Shodan 400 errors for unsupported types are suppressed; Shodan fields will be `unknown` for those rows

-

10. Relationship to Other Processes

The Tipper runs **independently** of the NOI forecasting pipeline but draws from the **same upstream OpDiv observation CSVs**.

```
OpDiv Observation Feed
        ↓  writes daily htoc_opdiv_obs_d{YYYYMMDD}.csv
        ├──→ NOI Forecasting (NextObservedIndicatorV3.0)
        └──→ Tipper (main.py)
                ↓  cross-references with ThreatConnect live query
                ↓  enriches with VirusTotal + Shodan
                ↓
        \\10.1.4.22\...\Tippers\tippers_by_partner_{YYYYMMDD}.xlsx
```

If OpDiv CSVs for the target date are missing, neither the NOI forecasting pipeline nor the Tipper will have current observation data.

-

11. Related Documents

`SOP_NextObservedIndicator_Forecasting.md` — shares the same OpDiv observation CSV inputs

`SOP_Daily_Reports_Consolidation.md` — downstream daily report process

`\\10.1.4.22\data\HTOC\HTOC_Reports\Tippers\` (output location)

`scripts/batch-processing-script/Tipper/utils/config.json` (ThreatConnect credentials)